

BEGINNER FRIENDLY SETUP

# 給初學者的 開發環境 設置教學

從已經有 Cursor 帳號開始，一步步接上 Git，理解 Gmail / GCP 的用途，最後選擇「地端 localhost」或「雲端全端」路線。

TODAY'S ROUTE

1

## Cursor 已經準備好

先把它當成「會陪你寫程式的編輯器」。

2

## 註冊 GitHub，連進 Cursor

讓程式有版本、有備份，也能被部署服務讀到。

3

## 準備 Gmail / Google 帳號

如果要使用 GCP，這就是登入雲端服務的鑰匙。

4

## 選擇你的全端路線

純地端 localhost，或部署到 GCP 雲端。

這份簡報不是要一次教完所有技術，而是先幫完全新手看懂「下一步要辦哪些帳號、為什麼要辦、辦完接到哪裡」。

## LESSON MAP

# 今天只走一條清楚的路

初學者最容易卡住的不是程式碼，而是「我到底要辦哪些帳號、接到哪裡」。

1

### Cursor 已有帳號

先把 Cursor 當成你的工作桌：開資料夾、問問題、改程式。

2

### 註冊 GitHub

大家常說「Git 帳號」，實務上通常是指 GitHub 帳號。

3

### 連進 Cursor

讓 Cursor 能讀取、發布、同步你的程式碼 repository。

4

### 選擇路線

看你只是本機練習，還是要把完整服務部署給別人用。

**localhost**  
地端

GCP  
雲端



## CURSOR FIRST

# Cursor 先當成「會陪你寫程式的編輯器」

你已經有帳號，所以第一步不是學完所有設定，而是知道它每天會幫你做哪三件事。

## 1 開一個專案資料夾

所有程式碼、設定、文件都放在同一個 workspace。

## 2 用 AI 問「下一步」

像問助教一樣：請它解釋檔案、修 bug、整理步驟。

## 3 把成果存進 Git

等一下接 GitHub 後，你的專案就能保存版本。



## WHY GIT

# Git 不是社群網站，是你的程式時光機

Git 負責記錄每次改動；GitHub 則像雲端倉庫，幫你把 Git 記錄放到網路上。

### 可以回到舊版本

改壞了不用恐慌，回頭找上一個穩定點。

### 可以備份到雲端

電腦壞掉，程式碼還在 GitHub。

### 可以讓服務部署

Cloud Run 等服務通常會從 repository 讀程式。

### 可以跟 AI 協作

每次改動可被比較、審查、回復。

#### GIT COMMITS



commit 1

**建立第一版網站**



commit 2

**加入登入功能，但還沒部署**



commit 3

**修好資料庫連線，準備發布**



## GITHUB ACCOUNT

# 註冊 GitHub：把程式碼放進雲端倉庫

這裡的重點不是學 Git 指令，而是先有一個能跟 Cursor 連接的帳號。

## 1 打開 github.com

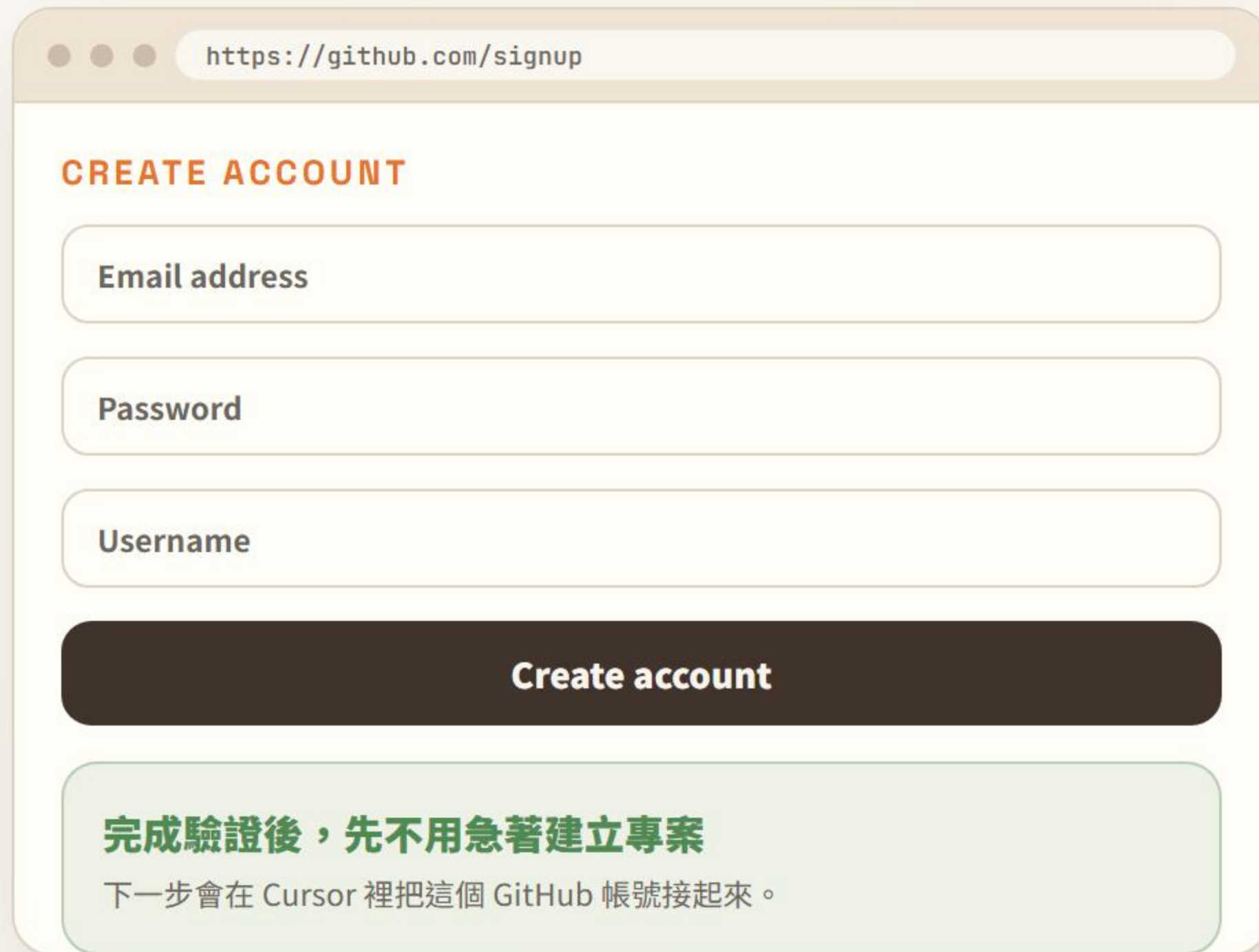
點 Sign up，照畫面填 email、密碼、使用者名稱。

## 2 完成 Email 驗證

GitHub 會寄驗證信，這一步做完才算帳號可用。

## 3 記住你的 username

之後 repository 網址、團隊邀請都會用到它。

A screenshot of the GitHub 'CREATE ACCOUNT' page. The browser's address bar shows 'https://github.com/signup'. The page has a white background with rounded corners. It features three input fields: 'Email address', 'Password', and 'Username'. Below these fields is a dark blue button labeled 'Create account'. At the bottom, there is a light green box containing the text '完成驗證後，先不用急著建立專案' and '下一步會在 Cursor 裡把這個 GitHub 帳號接起來。'.

https://github.com/signup

### CREATE ACCOUNT

Email address

Password

Username

Create account

**完成驗證後，先不用急著建立專案**

下一步會在 Cursor 裡把這個 GitHub 帳號接起來。

## CONNECT TO CURSOR

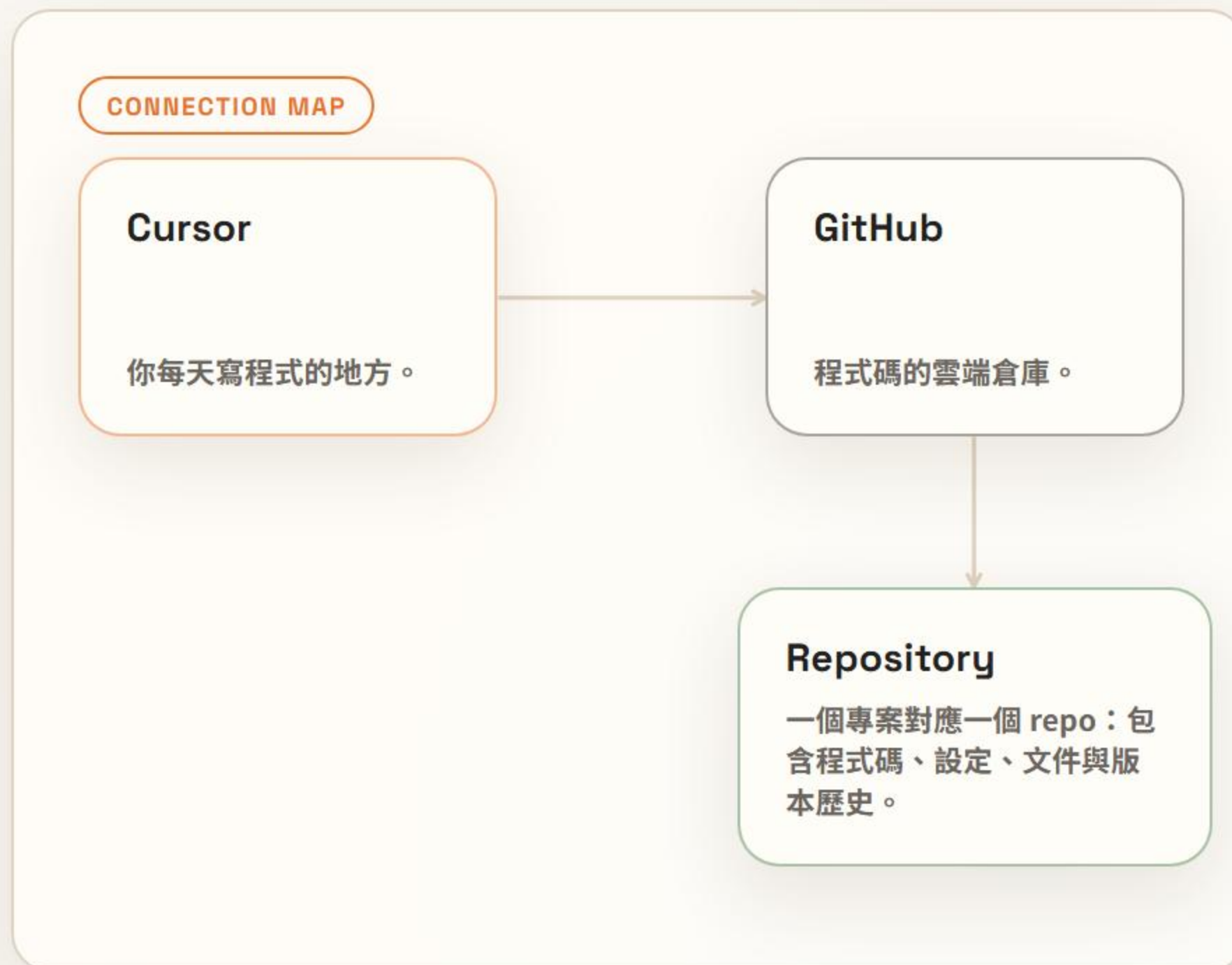
# 把 GitHub 連進 Cursor，專案才有去處

連接後，你可以 clone 別人的 repository，也可以把自己的專案 publish 到 GitHub。

```
git config --global user.name "你的名字"
```

```
git config --global user.email "你的 email"
```

這兩行只是告訴 Git「誰做了這次修改」。帳號登入則通常在 Cursor / GitHub 授權畫面完成。





## FULL-STACK PICTURE

# 你要打造的系統，長得像這樣

最關鍵的一點：前端和後端其實是「同一個 Next.js 專案」，一起部署。資料才分給 Firestore 與 Cloud SQL。

- 前端 + 後端 = 你同一個 app（用 App Hosting 部署）
- Firestore / Cloud SQL 負責把資料存起來
- 地端版可以把整個 app + 資料庫都放 localhost



## AUTO DEPLOY

# git push 出去，App Hosting 自動幫你部署

把 GitHub repo 接上 Firebase App Hosting 之後，你不用自己開伺服器、也不用手動上傳——每次 push 就會自動 build 並上線。



## 前端、後端是同一個 app

前端和後端都在同一個 Next.js 專案，App Hosting 一次就把它們部署完——所以不用自己設定 Cloud Run。

## 什麼時候才用 Cloud Run？

當你需要可以獨立執行、單獨擴展的服務（例如爬蟲、排程）時，才另外開 Cloud Run，再由後端呼叫它。



## CLOUD BUILDING BLOCKS

# 先認識這套雲端的幾個角色

不用今天全部設定好，先知道每個服務在系統裡負責哪一件事就好。

HOST · DEPLOY

## App Hosting

把你的 Next.js 前後端一起部署上線，git push 就自動更新。你的主後端就跑在這裡。

比喻：幫你顧好店面與後場的房東，你只管裝潢（寫 code）。

DOCUMENT DB

## Firestore

文件型 NoSQL，彈性資料、即時同步。前端也能直接連、後端也會用。

比喻：一疊可即時同步的資料夾，每份文件格式比較彈性。

RELATIONAL DB

## Cloud SQL

MySQL / PostgreSQL，適合表格、關聯、交易、報表。只在伺服器端使用。

比喻：Excel 表格升級成可被後端穩定查詢的資料庫。

STANDALONE SERVICE

## Cloud Run

把一個獨立服務包成容器單獨執行（例如爬蟲、排程），需要時由後端呼叫。

比喻：請一位專做某件事的外包小幫手，需要時才找它。

## GOOGLE ACCOUNT

# 如果要用 GCP， Gmail 就是進雲端的 鑰匙

GCP 是 Google Cloud Platform。通常你會用 Google 帳號登入，很多初學者最簡單就是準備 Gmail。

1

## 登入 Google Cloud Console

建立專案、啟用服務、管理帳單都在這裡。

2

## 建立 GCP Project

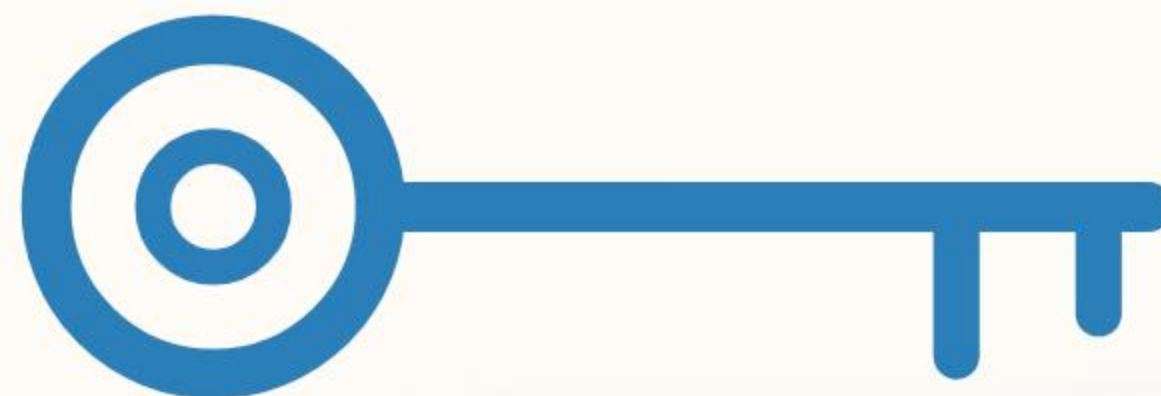
一個 project 像一個雲端工作區，裡面放 Cloud Run、Cloud SQL、Firestore。

3

## 管理權限與服務

未來也會用這個帳號決定誰能部署、誰能看資料。

ACCESS KEY



## Gmail / Google 帳號

不是因為寫 localhost 一定需要它，而是因為你要進入 GCP 的雲端服務時，需要一個 Google 身分。



## CHOOSE YOUR PATH

# 不要一開始就把自己丟進最複雜的架構

先問兩個問題：你只是自己練習，還是要讓別人連上？你需要真實雲端服務，還是本機跑起來就好？



## ROUTE A • LOCALHOST

# 地端全端：全部先在自己的電腦跑起來

這條路線最適合剛開始練習：沒有部署、沒有雲端帳單壓力，也不需要 Firebase。

**重點：**後端與資料庫都在本機，用 `localhost` 開。





ROUTE B · CLOUD

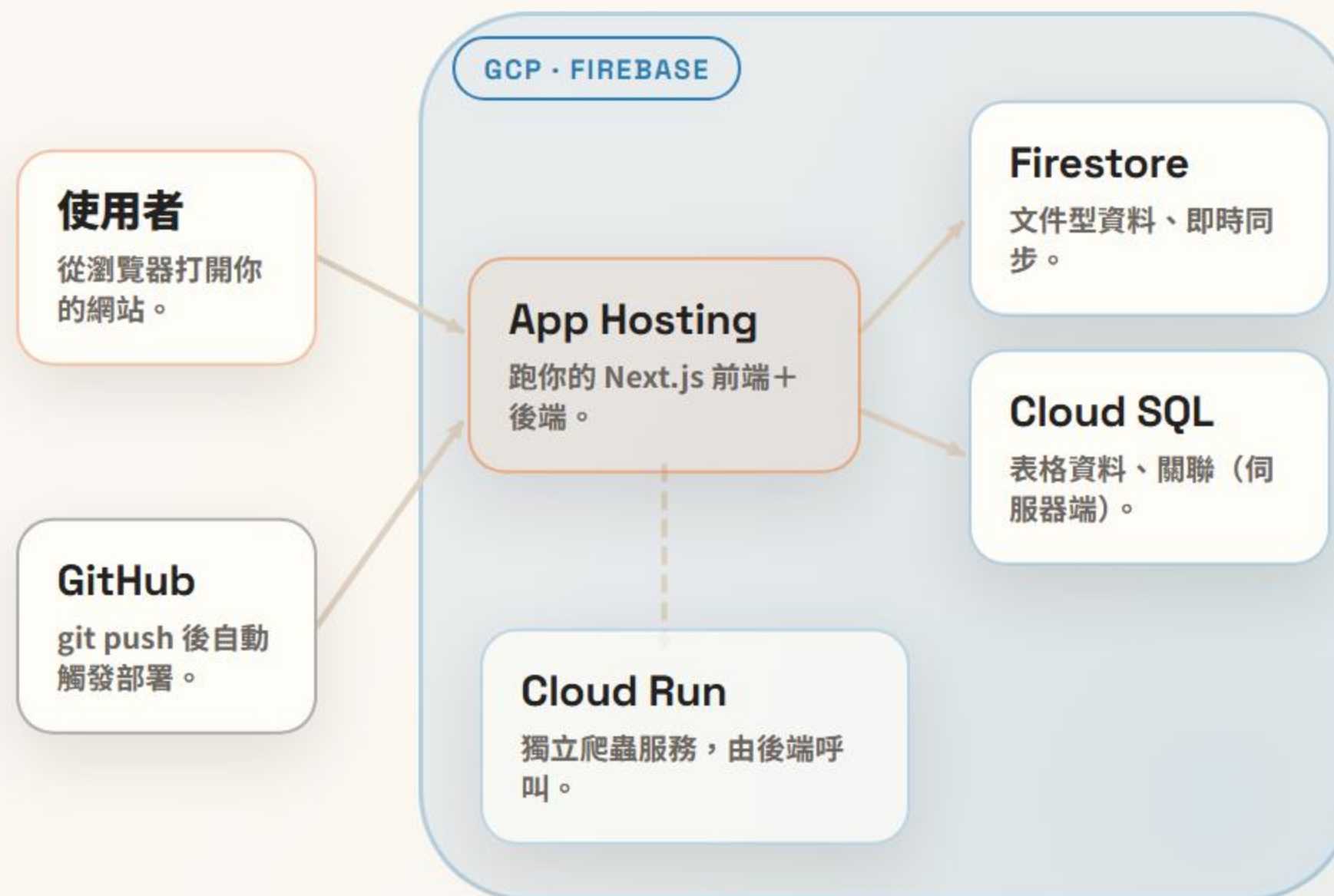
# 雲端全端：App Hosting 一鍵部署你的前後端

這條路線比較完整，也接近真實產品。push 後自動部署，資料交給 Firestore 與 Cloud SQL。你會需要 Gmail / Google 帳號進入 GCP。

需要 GitHub repository

需要 Gmail / Google 帳號

需要理解 App Hosting + 資料庫



QUICK DECISION

最後，用這張表選路線

不要為了「看起來完整」就直接上雲端。先選最符合當下目標的路線。

| 判斷項目          | 路線 A · 地端 localhost             | 路線 B · GCP 雲端全端                             |
|---------------|---------------------------------|---------------------------------------------|
| 適合誰           | 剛開始學、自己練習、先把功能跑通。               | 要讓別人使用、展示作品、接近正式產品。                         |
| 需要帳號          | Cursor + GitHub 就很夠。            | Cursor + GitHub + Gmail / Google 帳號。        |
| 資料庫位置         | 在本機，或先用假資料練習。                   | Firestore / Cloud SQL 放在 GCP。               |
| 是否需要 Firebase | 不需要。後端與資料庫都在本機即可。               | 需要。用 App Hosting 部署、Firestore 存資料。          |
| 部署方式          | 在本機 npm run dev，用 localhost 開。  | git push 後 App Hosting 自動 build + 上線。       |
| 下一步           | 先建立第一個 repo，讓專案可以在 localhost 跑。 | 建立 GCP / Firebase 專案，把 repo 接上 App Hosting。 |

如果你完全新手

先走地端路線。你會更快看到成果，也比較不會被雲端設定嚇到。

如果你要做真實服務

再進雲端路線。把 repo 接上 App Hosting，git push 就自動部署，搭配 Firestore / Cloud SQL。



## WORK MODES

# 兩種模式：先想清楚，再動手

Cursor 有兩種工作方式。學會在對的時機切換，是把 AI Coding 做準的第一步。

### THINK FIRST

## Plan 模式

唯讀模式，先一起把做法想清楚。Cursor 會問問題、提出計畫，但不會改你的檔案。

適合什麼時候

- 需求比較大，或有好幾種做法
- 想先釐清方向再動手
- 需要一份能先確認的 Plan

### EXECUTE

## Agent 模式

執行模式，會實際讀寫檔案、跑指令，把任務直接做出來。

適合什麼時候

- 方向已經很清楚
- 小修改、單一明確任務
- 直接 build，馬上看結果

## GOOD PROMPTS

# 把需求講清楚，AI 才做得準

AI 不會通靈。資訊越完整，它越能一次到位。一個好需求通常包含這四件事：

01

### 目標

你想達成什麼結果。

02

### 脈絡

相關檔案（用 @ 帶入）、限制與目前狀況。

03

### 成功標準

怎樣才算完成，而且能被驗證。

04

### 範例與邊界

給個例子，並說清楚「不要動什麼」。

× 不夠清楚

「幫我修一下這個。」

沒說哪個檔、預期行為，也沒有邊界，AI 只能用猜的。

✓ 清楚具體

「@login.tsx 密碼錯誤時沒有提示。請在錯誤時顯示紅字訊息，成功則導向 /home，不要動到註冊流程。」

有檔案、有預期行為、有邊界，AI 一次就能做對。



## CLARIFY THEN BUILD

# 來回釐清，先有精準的 Plan 再 Build

別急著叫它寫。先把需求磨清楚、定好計畫，執行就會又快又準。

**重點**

在 **Plan 模式** 反覆對話，把需求磨成精準的 Plan，再切 **Agent build**。改一點、驗一點，比一次做完更穩。

DO IT WELL

# 把 AI Coding 做得又快又準

把前面幾頁串起來，這六個習慣會讓你少走很多冤枉路。

01

## 小步快跑

一次一個明確任務，別一次塞十件事。

02

## 給足脈絡

用 @ 帶相關檔案、貼上錯誤訊息、說清楚現況。

03

## 設好成功標準

能驗證（跑得起來、測試過、畫面對）才算完成。

04

## 檢視每次改動

看 diff、理解了再接受，別盲目全部套用。

05

## 卡住就換策略

回到 Plan、縮小範圍，或請它先解釋給你聽。

06

## 先能動，再優化

先把功能跑通，再回頭美化與重構。